

CHAROTAR UNIVERSITY OF SCIENCE & TECHNOLOGY

Faculty of Technology & Engineering

Devang Patel Institute of Advance Technology and Research (DEPSTAR)

Advanced Web Development and Frameworks (ITUE301)

Practical File – ODD Semester 2025–26

Student ID: 24DIT020

Student Name: Gondha Stuti

Practical 4: Building a RESTful API with Node.js and Express

Project Definition:

Build a Node/Express backend for a Task Management system.

Implement REST endpoints for creating, reading, updating, and deleting tasks (in-memory array, no database yet).

Apply a request logging middleware that logs method, URL, and timestamp for every incoming request.

Apply a global error handling middleware as the last middleware in the pipeline.

Use correct HTTP status codes (200, 201, 404, 500) for each response.

Code:

```
const express = require("express");
```

```
const app = express();
```

```
const PORT = 3000;
```

```
app.use(express.json());
```

```
let tasks = [];
```

```
app.use((req, res, next) => {
```

```
  console.log(
```

```
    `${new Date().toLocaleString()} | ${req.method} | ${req.url}`
```

```
  );
```

```
  next();
```

```
});
```

```
app.get("/tasks", (req, res) => {
```

```
  res.status(200).json(tasks);
```

```
});
```

```
app.post("/tasks", (req, res) => {
```

```
  const task = {
```

```
    id: Date.now(),
```

```
    title: req.body.title,
```

```
    completed: false
```

```
  };
```

```
  tasks.push(task);
```

```
  res.status(201).json(task);
```

```
});
```

```
app.put("/tasks/:id", (req, res, next) => {
```

```
    const id = Number(req.params.id);
```

```
    const task = tasks.find(t => t.id === id);
```

```
    if (!task) {
```

```
        const err = new Error("Task Not Found");
```

```
        err.status = 404;
```

```
        return next(err);
```

```
    }
```

```
    task.title = req.body.title || task.title;
```

```
    if (req.body.completed !== undefined)
```

```
        task.completed = req.body.completed;
```

```
    res.status(200).json(task);
```

```
});
```

```
app.delete("/tasks/:id", (req, res, next) => {
```

```
    const id = Number(req.params.id);
```

```
const index = tasks.findIndex(t => t.id === id);

if (index === -1) {

    const err = new Error("Task Not Found");
    err.status = 404;
    return next(err);

}

tasks.splice(index, 1);

res.status(200).json({
    message: "Task Deleted"
});

});

app.use((err, req, res, next) => {

    res.status(err.status || 500).json({

        error: err.message || "Internal Server Error"

    });

});

app.listen(PORT, () => {
```

```
console.log(`Server running at http://localhost:${PORT}`);  
  
});
```

GET:

The screenshot shows a web client interface with a GET request to `http://localhost:3000/tasks`. The response is a JSON array containing one object. The status is 200 OK, the size is 74 Bytes, and the time is 21 ms.

Request: GET `http://localhost:3000/tasks` Send

Query Headers² Auth Body Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

1

Status: 200 OK Size: 74 Bytes Time: 21 ms Response

```
1  [  
2    {  
3      "id": 1786003792356,  
4      "title": "Complete Node Practical",  
5      "completed": false  
6    }  
7  ]
```

Copy

Response Chart ↗

POST:

POST ⌵ http://localhost:3000/tasks Send

Query Headers ² Auth **Body ¹** Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

```
1 {
2   "title": "Complete Node Practical"
3 }
```

Status: **201 Created** Size: **72 Bytes** Time: **52 ms** Response ⌵

```
1 {
2   "id": 1786003792356,
3   "title": "Complete Node Practical",
4   "completed": false
5 }
```

Copy

PUT:

PUT ⌵ http://localhost:3000/tasks/1786003792356 Send

Query Headers ² Auth **Body ¹** Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

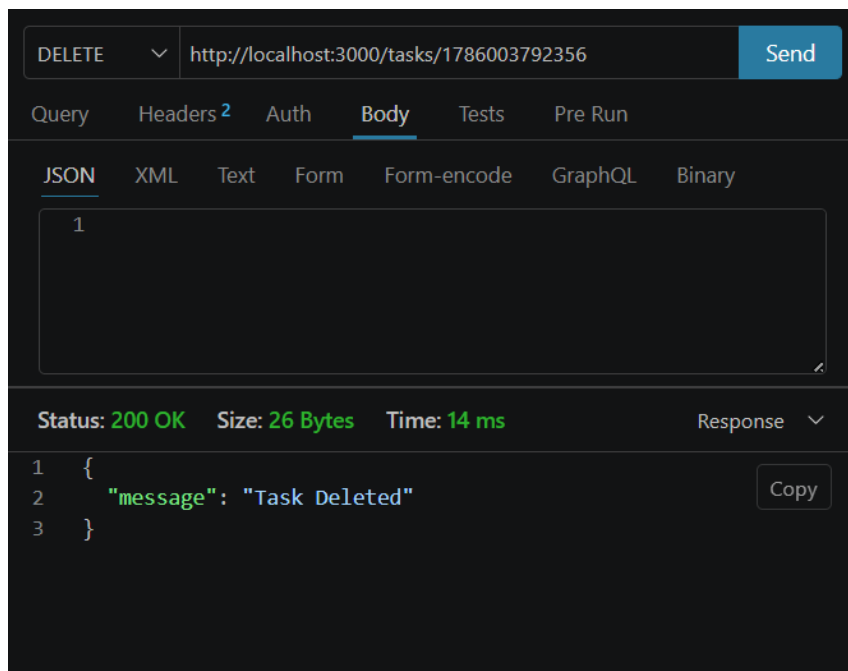
```
1 {
2   "title": "Complete Express Practical",
3   "completed": true
4 }
```

Status: **200 OK** Size: **74 Bytes** Time: **13 ms** Response ⌵

```
1 {
2   "id": 1786003792356,
3   "title": "Complete Express Practical",
4   "completed": true
5 }
```

Copy

DELETE:



Supplementary Problem:

Add a 404 handler for undefined routes that returns a structured JSON response.

